BOSS-KAMPF 3 · Welt 3 · DS 23 · 60 min · Hilfsmittel: Werkzeugkarten

W3·Boss – Die Kristallhöhle

Erreichbar: 24 BE · Papier-Compiler zu e) im Anschluss · Zusätzlich im Lab: **Online-Boss „Das Echo der Schlucht“**

Tief unter dem Labyrinth liegt die Kristallhöhle. Die Zwerge der Bergmannszunft bauen dort leuchtende Kristalle ab, ordnen sie in Vitrinen und lagern sie sortiert ein. Ihre Werkzeuge sind rekursive Operationen der Klasse **Zwergenwerkstatt** (Abbildung 2) und der Klasse **Hoehle** (Abbildung 4) – doch nicht alle arbeiten so, wie die Zwerge glauben. Zeig, dass du Rekursion verfolgen, analysieren und selbst entwerfen kannst!

Hinweise zur Bearbeitung

- Bearbeite alle Teilaufgaben **ohne Gerät** auf dem Antwortbogen am Ende dieses Hefts. Erlaubt sind nur deine Werkzeugkarten.
- Implementierungen schreibst du in **Java**, Struktogramme nach den Abi-Konventionen (Kopf mit Signatur, ←, „gib … zurück“; ein rekursiver Aufruf ist eine normale Anweisung).
- Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.
- Zeitbudget wie im Abitur: ca. 2,5 Minuten pro BE. Die BE stehen rechts neben jeder Teilaufgabe.

Papier-Compiler und Online-Boss

Deine Lösung zu **Aufgabe e)** tippst du direkt nach dem Boss-Kampf **unverändert** in der Online-IDE ab und startest die Tests im Testrunner. Kreuze im Antwortbogen an, ob alle Tests beim ersten Versuch oder erst nach einer Korrektur grün waren. Typische Fallen: Rand nicht vor jedem Feldzugriff geprüft, erst nach den Selbstaufrufen markiert.

Zusätzlich im Lab: Online-Boss „Das Echo der Schlucht“ (25 min, Auswertung am Ende, der beste Versuch zählt) – Punkte gibt es dort automatisch.

Aufgaben

a) Die Zwerge ordnen die Kristalle einer Vitrine nach ihrer Leuchtkraft mit der rekursiven Operation **ordne** (Abbildung 3). Die Reihung vitrine ist in Abbildung 1 angegeben; ihr Parameter heißt in ordne r.

Dokumentieren Sie im Antwortbogen alle Aufrufe von ordne, die bei der Verarbeitung des Aufrufs `ordne(vitrine, 0, 2)` erfolgen, in der Reihenfolge ihres Beginns. Notieren Sie zu jedem Aufruf die Rekursionstiefe und die Belegung der Reihung zum Zeitpunkt des Aufrufs. Geben Sie die Belegung nach dem Ende des Erstaufrufs an. **[4 BE]**

b) Die Anzahl der Aufrufe von ordne (einschließlich des Erstaufrufs) hängt nur von der Anzahl n der Elemente im Bereich li bis re ab, nicht von deren Werten.

Geben Sie mithilfe Ihrer Dokumentation aus a) die Anzahl der Aufrufe für n = 1, n = 2 und n = 3 an. Ermitteln Sie die Anzahl der Aufrufe für n = 4.

Für n = 8 sind es 124, für n = 16 bereits 1219 Aufrufe; mergeSort wird bei 8 Elementen 15-mal und bei 16 Elementen 31-mal aufgerufen. Ein Zwerg behauptet: „ordne teilt die Reihung wie Mergesort in zwei Hälften, also ist ordne genauso effizient.“ Beurteilen Sie diese Aussage. **[3 BE]**

c) Erläutern Sie anhand der Operation ordne das Prinzip rekursiver Operationen.

Begründen Sie, dass nach der Verzweigung mit der Bedingung r[m] > r[re] der größte Wert des Bereichs li bis re an der Position re steht.

[3 BE]

d) Die Operation **anzahlHell** soll die Anzahl der Kristalle in r[von] … r[bis] liefern, deren Leuchtkraft mindestens grenze beträgt (es gilt von ≤ bis). Sie soll nach dem Prinzip Teile und herrsche arbeiten: Ein Bereich mit mehr als einem Element wird in der Mitte geteilt, beide Teile werden rekursiv bearbeitet und die Teilergebnisse zusammengeführt. Schleifen dürfen nicht verwendet werden.

Beispiel: `anzahlHell(leuchtkraft, 10, 0, 7)` liefert mit der Belegung aus Abbildung 1 den Wert 4.

Entwickeln Sie ein Struktogramm für die Operation anzahlHell.

[4 BE]

e) Die Höhlenkarte wird durch ein Objekt der Klasse **Hoehle** verwaltet (Abbildung 4). feld[z][s] ist das Feld in Zeile z und Spalte s: '#' Fels, '.' Gang, 'K' Kristall, '-' abgebauter Kristall. Kristalle, die oben, rechts, unten oder links (nicht diagonal) aneinandergrenzen, bilden eine Kristallader.

Die Operation **abbauen** baut die gesamte Kristallader ab, zu der das Feld (z|s) gehört: Alle ihre Kristalle werden zu '-', geliefert wird die Anzahl der abgebauten Kristalle. Liegt (z|s) außerhalb der Höhle oder befindet sich dort kein Kristall, wird 0 geliefert und nichts verändert. Beispiel: Für die Höhle aus Abbildung 4 liefert `abbauen(2, 3)` den Wert 6; danach sind die Felder (0|4), (0|5), (1|5), (2|3), (2|4) und (2|5) abgebaut.

Implementieren Sie die Operation abbauen rekursiv in Java.

[5 BE]

f) Im Lager liegen die Kristalle aufsteigend nach Leuchtkraft sortiert (Reihung lager, Abbildung 1). Die rekursive Operation **finde** (Abbildung 5) soll den Index eines Kristalls mit der Leuchtkraft x im Bereich r[lo] … r[hi] liefern bzw. −1, falls es keinen gibt. `finde(lager, 14, 0, 5)` liefert korrekt den Wert 4. Bei bestimmten Eingaben arbeitet die Operation jedoch fehlerhaft.

Erläutern Sie am Aufruf `finde(lager, 17, 0, 5)`, wie es zu dem Fehler kommt und welche Folge er hat. Verändern Sie das Struktogramm so, dass der Fehler behoben wird, und begründen Sie, dass die veränderte Operation für jede Eingabe endet.

[5 BE]

Summe: 24 BE

Material

zu den Aufgabenteilen a), b), d) und f)

Index	0	1	2
vitrine	9	6	4

Index	0	1	2	3	4	5	6	7
leuchtkraft	7	12	3	15	9	12	4	10

Index	0	1	2	3	4	5
lager	2	5	8	11	14	17

Abbildung 1: Belegung der Reihungen vitrine, leuchtkraft und lager (jeweils Reihung von Ganzzahl, Leuchtkraft der Kristalle). lager ist aufsteigend sortiert.

Signatur	Beschreibung
ordne(r: Reihung von Ganzzahl, li: Ganzzahl, re: Ganzzahl): nichts	ordnet die Werte r[li] … r[re] (Abbildung 3)
anzahlHell(r: Reihung von Ganzzahl, grenze: Ganzzahl, von: Ganzzahl, bis: Ganzzahl): Ganzzahl	Aufgabenteil d)
finde(r: Reihung von Ganzzahl, x: Ganzzahl, lo: Ganzzahl, hi: Ganzzahl): Ganzzahl	Aufgabenteil f), Abbildung 5

Abbildung 2: Übersicht über die Klassenoperationen der Klasse Zwergenwerkstatt

zu den Aufgabenteilen a) bis c)

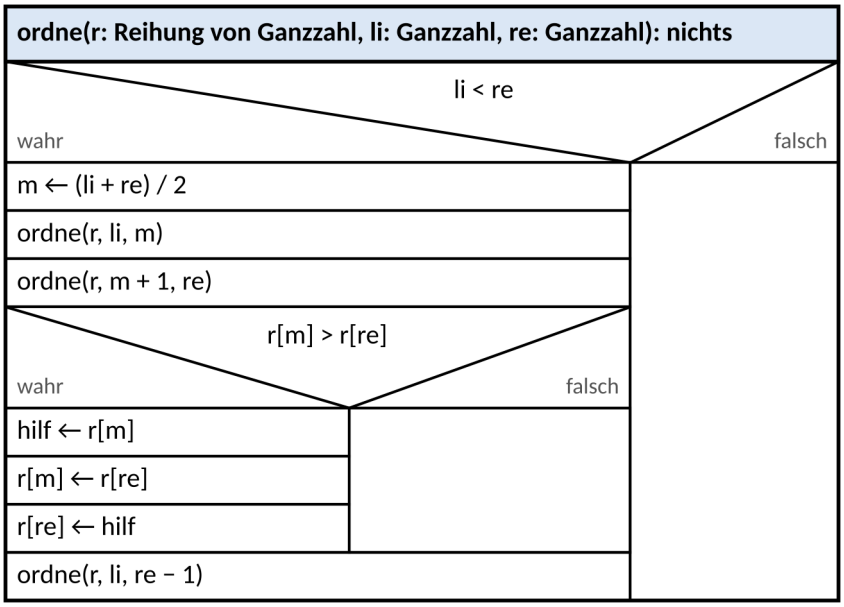


Abbildung 3: Struktogramm der rekursiven Operation ordne. Hinweis: Die Division von Ganzzahlen ist als Division ohne Rest zu verstehen.

zu Aufgabenteil e)

Hoehle
- feld: Reihung von Reihung von Zeichen
+ c Hoehle(zeilen: Reihung von Zeichenkette) + istImFeld(z: Ganzzahl, s: Ganzzahl): Wahrheitswert + getFeld(z: Ganzzahl, s: Ganzzahl): Zeichen + abbauen(z: Ganzzahl, s: Ganzzahl): Ganzzahl ...

	s = 0	s = 1	s = 2	s = 3	s = 4	s = 5
z = 0	K	K	#	.	K	K
z = 1	#	K	.	.	#	K
z = 2	.	K	#	K	K	K
z = 3	K	.	.	#	.	.

grau: Fels '#' · blau: Kristall 'K' · weiß: Gang '.'

Abbildung 4: Klassenkarte der Klasse Hoehle und Belegung von feld für die Höhle aus Aufgabenteil e) (feld[z][s]: Zeile z, Spalte s; die Höhle ist nicht von Fels umgeben)

zu Aufgabenteil f)

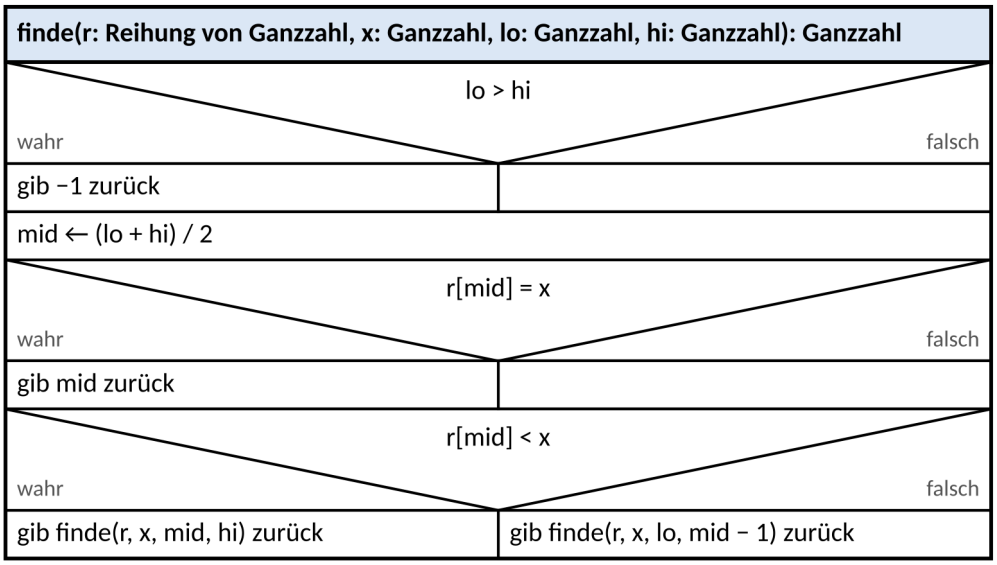


Abbildung 5: Struktogramm der rekursiven Operation finde. Hinweis: Die Division von Ganzzahlen ist als Division ohne Rest zu verstehen.

Antwortbogen

a) Rekursionsverlauf von ordne(vitrine, 0, 2) – Parameter r verweist auf vitrine **[4 BE]**

Rekursionstiefe *)	Aufruf	r[0]	r[1]	r[2]
0	ordne(r, 0, 2)	9	6	4

*) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.

Belegung nach dem Ende des Erstaufrufs: r[0] = _____ r[1] = _____ r[2] = _____

b) Anzahl der Aufrufe · Beurteilung **[3 BE]**

n = 1: _____ n = 2: _____ n = 3: _____ n = 4: _____

c) Prinzip rekursiver Operationen · Begründung **[3 BE]**

d) Struktogramm der Operation anzahlHell **[4 BE]**

Zeichenfläche für das Struktogramm (Kopf mit Signatur nicht vergessen)

e) Implementierung von abbauen – Papier-Compiler: sauber und lesbar schreiben! **[5 BE]**

f) Fehler in finde: Verlauf, Folge, veränderte Stelle im Struktogramm, Begründung **[5 BE]**

Veränderung im Struktogramm (es genügt, den veränderten Block mit seiner Umgebung zu zeichnen):

Auswertung (Peer-Korrektur nach Kernpunkten)

Teilaufgabe	a)	b)	c)	d)	e)	f)	Summe
max. BE	4	3	3	4	5	5	24
erreicht							

Korrigiert von: _____ Papier-Compiler e): ☐ grün beim 1. Versuch ☐ grün nach Korrektur ☐ nicht grün